

Wiki Jurídica — Relatório de Engenharia v2 (IA-first, hardware próprio)

Data-base: 15/09/2026. Núcleo Go, ML em Python, engenharia por Claude Code.
Servidor Queimados/RJ: Ryzen 7 9800X3D, RTX 5060 Ti 16 GB (Blackwell sm_120), B850, 32→96/192 GB DDR5, NVMe 4 TB.

TL;DR

- **Separe fisicamente os dois planos.** 100k+ citações/dia é problema de CDN e formato, não de GPU: HTML ≤50 KB + `.md` + JSON 100% cacheáveis, ETag/304, IndexNow e feeds de delta resolvem o volume no edge; a GPU fica 100% para inferência paga e treino noturno.
- **O bot próprio nasce por poda+destilação, não por SFT.** Alvo: um jurídico 4–8B destilado de 14–32B via Minitron (prune+distill) + on-policy distillation (Thinking Machines, out/2025), servido a ~3 bpw EXL3 nos 16 GB, com KV FP8, recompensa verificável por citação (URN resolve no banco + NLI confirma o trecho).
- **A camada M2M já tem trilhos prontos em Go.** x402 (Go oficial no monorepo Coinbase), Cloudflare Pay-per-Crawl/Monetization Gateway (402+x402, já no seu Cloudflare), L402/Aperture (Go, Lightning) e MCP Go SDK v1.x estável — dá para faturar centavos por chamada/crawl sem stack de pagamento próprio.

Sumário executivo — as 12 decisões-chave

1. **Arquitetura dois planos:** conteúdo (edge/CDN) vs. inferência (GPU). Nunca deixar crawler tocar a origem dinâmica.
2. **Formato agente-first:** negociação `Accept: text/markdown + .md / .json` canônicos por página + "answer unit" com URN LexML e âncoras art./§/inciso.
3. **Serving:** vLLM como motor primário em sm_120 (NVFP4/FP8 nativos); llama.cpp para MoE com offload de experts e hot-swap de LoRA; llama-swap (Go) como roteador.
4. **Modelo de trabalho:** 14B FP8 residente + 27–32B EXL3 ~3 bpw sob demanda; cascata pequeno→MoE por incerteza.
5. **Quantização:** EXL3 (trellis/QTIP) como default de qualidade×bpw; NVFP4 para throughput em Blackwell; KV FP8; embeddings int8/Matryoshka.
6. **Destilação:** Minitron prune+distill (14–32B→4–8B) + on-policy distillation; professores por offload noturno; RLVR/GRPO em 16 GB via Unsloth.

7. **Fusão:** MergeKit (della/sce/karcher/model_stock) + X-LoRA/S-LoRA como "MoE de LoRAs" por ramo do direito.
8. **Captura:** DJEN/Comunica (API pública, texto), dados abertos STJ (integras JSON), inteiro teor STF (PDF), LexML SRU/OAI-PMH + urn:lex:br, Planalto HTML /ccivil_03/ ; DataJud só metadados.
9. **Índice:** Postgres + pgvector/pgvectorscale (híbrido BM25+denso+rerank), Apache AGE p/ grafo lei ↔ decisão ↔ súmula ↔ tema com vigência.
10. **Reuso Go:** hugot (ONNX: embeddings/NER/rerank sem Python), llama.cpp via cgo ou sidecar, MCP go-sdk oficial, River sobre Postgres, colly/rod.
11. **M2M:** ledger de créditos em Go, x402 + Pay-per-Crawl + L402; catálogo publicado no MCP Registry e x402 Bazaar.
12. **Rede social IA:** identidade Ed25519 + Web Bot Auth (RFC 9421), MCP tools p/ registro/ações, reputação por citações verificadas, mídia local (TTS/vídeo) no NVMe+CDN.

S1. Arquitetura "dois planos" para 100k+ citações/dia sem tocar a GPU

Princípio. O baseline atual (417 cliques, 20,8k impressões, ~23k citações por IAs em <30 dias, ainda com IA em CPU) mostra que o gargalo para 100k citações/dia é **entrega de bytes**, não compute. Bots de alta frequência (GPTBot, OAI-SearchBot, ClaudeBot, PerplexityBot, Bingbot, Google-Extended) requisitam páginas repetidamente; cada requisição precisa ser resolvida no edge Cloudflare com custo de origem ~zero.

Plano de conteúdo (100% edge)

- **Cache agressivo:** Cache-Control: public, max-age=300, stale-while-revalidate=86400, stale-if-error=604800; cache-everything via Cloudflare; **cache tags** por norma/decisão para *purge* cirúrgico por API a cada publicação.
- **Condicional para crawlers:** ETag forte + honrar If-None-Match → **304** para o alto volume de recrawl (a maior economia real de banda de upload doméstico).
- **Compressão/transporte:** Brotli nível 5–6 para HTML, zstd onde suportado; **HTTP/3; Early Hints (103)** com Link: rel=preload.
- **Descoberta:** sitemaps segmentados (por tribunal/ano/ramo), **IndexNow em lote** a cada publicação, WebSub/RSS + **JSONL de deltas diários**.

Exemplo de cabeçalhos de uma página "answer unit":

```
HTTP/3 200
content-type: text/html; charset=utf-8
vary: accept
etag: "au-8f3a1c-2026-09-15"
```

```
cache-control: public, max-age=300, stale-while-revalidate=86400
link: </au/stf-tema-1234.md>; rel="alternate"; type="text/markdown"
x-canonical-urn: urn:lex:br:supremo.tribunal.federal:tema;1234
x-verified-at: 2026-09-15T00:00:00Z
```

Engenharia da origem em Go (link doméstico com upload limitado)

- **Coalescência de requisições** (`singleflight`) para render sob demanda; **ristretto** ou **groupcache** como cache in-process de páginas pré-renderizadas.
- **Pré-render** batch noturno: o modelo local gera as "answer units" e grava HTML+MD+JSON estáticos; a origem serve arquivos, não templates.
- **Bot lanes**: fila de prioridade por user-agent **verificado** (Web Bot Auth / Verified Bots) — crawlers verificados recebem 304/arquivo estático com prioridade; tráfego não verificado entra em fila de menor prioridade e rate shaping.
- **Cloudflare Markdown for Agents** (GA 12/02/2026): `Accept: text/markdown` converte HTML→MD no edge, com `content-type: text/markdown`, `vary: accept` e header `x-markdown-tokens` (contagem estimada de tokens). Exemplo oficial Cloudflare: HTML 16.180 tokens → MD 3.150 tokens. Claude Code e OpenCode já enviam `Accept: text/markdown`. Como o cache do edge é chaveado por `Accept`, HTML e MD ficam em entradas separadas — servir o `.md` próprio pré-gerado dá controle total de front-matter e âncoras (o converter do edge é fallback).

Negociação de conteúdo e "answer unit"

Cada página canônica traz um bloco citável determinístico:

- resumo canônico + **IDs LexML** (`urn:lex:br`), âncoras por artigo/§/inciso, **data de verificação**, **versão da norma** (vigência).
- **JSON-LD**: `Legislation`, `LegislationObject`, `DefinedTermSet`, `Dataset`, `WebAPI`, `FAQPage`.
- **NLWeb** (MCP + Schema.org) como camada de descoberta conversacional do próprio site.

Aviso baseado em evidência 2025–2026 sobre schema: não superestimar JSON-LD como alavanca de citação. O teste controlado da Ahrefs (ago/2025–mar/2026, 1.885 páginas com schema vs ~4.000 de controle) **não** encontrou ganho de citação por marcação estruturada — AI Overviews caíram 4,6% (estatisticamente significativo), AI Mode +2,4% e ChatGPT +2,2% (ambos não significativos). O sinal positivo real vem de **GEO** (estudo de Princeton): inclusão de **estatísticas (+37%)**, **citações inline (+40%)** e **citações de especialista (+22%)** elevam a taxa de menção. Ou seja: o "answer unit" deve priorizar densidade de dados numéricos, citações inline com URN e atribuição a fonte oficial — não schema por schema.

Instrumentação

- **Logpush** (Cloudflare) → Vector → Postgres → Grafana; ou GraphQL Analytics API → Postgres. Dimensões: bot, página, status (200/304), citação, referral.
- **Atribuição de referrals**: casar `Referer` /UTM de chatgpt.com, perplexity.ai, claude.ai, copilot.microsoft.com, gemini.google.com; correlacionar com o relatório "AI Performance" do Bing (fonte do ChatGPT/Copilot via índice Bing).
- **O que faz citar (evidência 2026)**: o estudo Wellows ("89% of What ChatGPT Cites, Perplexity Never Touches", jan–jun/2026) analisou **22.749.707 citações em 1.146.483 perguntas** e concluiu que **79,6% das fontes citadas aparecem em apenas um dos cinco motores** e **só 0,31% (27.097 pares) apareceram nos cinco**; a página exata coincide 6,8% e a marca 30,3%. Implicação de engenharia: **cobrir todos os motores separadamente** (Bing p/ ChatGPT/Copilot; RAG próprio p/ Perplexity; AI Overviews/AI Mode; Claude; Gemini), porque não há fórmula única. Perplexity roda RAG de ~6 estágios (relevância semântica, frescor, qualidade estrutural, autoridade, engajamento) e cita muitas fontes por consulta; [Authoritytech](#) ChatGPT cita menos mas absorve mais fundo; Grok pondera frescor e presença no X.

Brainstorm de inovação (S1)

- **Citation receipts assinados (Ed25519)**: endpoint retorna, junto ao trecho, um recibo `{urn, hash_sha256, versao, verified_at, sig}` que o agente guarda como prova de fonte. Content-addressed: o ID da resposta é o hash do trecho canônico.
- **Endpoint `/verify`**: o agente envia um hash/URN e recebe o trecho canônico + status de vigência + assinatura — vira ferramenta anti-alucinação que **outras IAs pagam para chamar** (liga direto ao S8).
- **Feeds de delta JSONL diários e knowledge packs** (Parquet + embeddings alinhados ao índice) baixáveis — produto M2M nativo.

S2. Captura de texto integral jurídico em escala

Mapa de fontes e estratégia técnica

Fonte	Formato	Como obter INTEGRAL
DJEN/Comunica (CNJ)	JSON (API pública <code>comunica.pje.jus.br</code>)	despachos, decisões interlo publicados por lei (art. 13 Ri OAB/processo/tribunal/dat

Fonte	Formato	Como obter INTEGRAL
STJ dados abertos (dadosabertos.web.stj.jus.br)	JSON (CKAN API)	dataset com íntegras e met
STF inteiro teor	HTML SPA + PDF	portal.stf.jus.br/juri → PDF do inteiro teor; SPA endpoints JSON
LexML	XML (SRU) + OAI-PMH	SRU por URL retorna metad: no publicador
Planalto	HTML estático /ccivil_03/	texto integral direto no HTM /leis/1NNNNN.htm, /_at
Câmara/Senado	JSON/XML (dados abertos)	APIs de proposições/norma
TST/TRFs/TJs (e-SAJ/PJe/Eproc)	HTML+PDF	consultas públicas; inteiro ti
DataJud (CNJ)	JSON (Elasticsearch POST, APIKey)	só metadados/movimentaç

Regra de ouro: onde a API dá só ementa/metadados, o **inteiro teor vem do PDF do próprio tribunal** (inteiro teor STF, e-SAJ/PJe) ou do HTML do publicador (Planalto para normas). O pipeline sempre resolve URN→publicador→PDF/HTML→texto. Concretamente: LexML SRU devolve metadados + URL; o URN urn:lex:br:federal:lei:1993-06-21;8666 resolve em lexml.gov.br/urn; a íntegra da Lei 8.666 está no HTML do Planalto (/ccivil_03/leis/18666...). Para acórdãos STF, cada resultado da base "Inteiro Teor de Acórdãos" expõe um ícone PDF com o texto completo (busca full-text disponível para publicados a partir de 2012; PDF recuperável para publicados após 06/07/1950).

Pipeline de ingestão

1. **Crawling distribuído:** Go **colly** (estático) + **rod** (headless Chrome quando há JS); filas com **River** (durabilidade sobre Postgres) ou **NATS JetStream**; rate shaping por domínio; retomada por checkpoint; dedup por hash de conteúdo.

2. **PDF→texto: Docling** (IBM) orquestrando OCR; na 5060 Ti, VLM-OCR de 1–3B (PaddleOCR-VL-1.6 ~2 GB FP16, DeepSeek-OCR, Granite-Docling) via servidor vLLM local. Benchmarks de referência: PaddleOCR clássico ~380 páginas/min na RTX 5090; [GigaGPU](#) na 5060 Ti espere fração proporcional à banda (448 vs 1792 GB/s) **[est. ~90–110 páginas/min PaddleOCR clássico]**. Docling:

```
ThreadedPdfPipelineOptions(ocr_batch_size=64, layout_batch_size=64) + AcceleratorDevice.CUDA
```

; rodar **N processos worker** (a GIL limita concorrência interna — batch size sozinho não move throughput) para saturar a GPU.
3. **Segmentação**: ementa / relatório / voto / dispositivo por regex+layout.
4. **NER pt-BR em ONNX (via hugot, sem Python)**: BERTimbau/Albertina/LeNER-Br para partes, relator, órgão, classe, datas.
5. **Dedup**: MinHash/SimHash (acórdãos replicados entre bases).
6. **Normalização de citações**: para URN LexML, numeração única CNJ, súmula/tema.
7. **Grafo**: Postgres + **Apache AGE** — lei ↔ decisão ↔ súmula ↔ tema com **vigência temporal**.

Dimensionamento do índice (NVMe 4 TB)

- Alvo exemplo: 10M decisões × ~8 chunks × 1024-dim. **pgvector/pgvectorscale** (StreamingDiskANN) mantém índice em disco com quantização; **quantização de vetores int8** corta 4x o footprint; **binary/Matryoshka** para primeiro estágio.
- **Busca híbrida**: BM25 pt-BR (Postgres FTS ou Bleve/Meilisearch) + denso (pgvector) + **rerank** Qwen3-Reranker-0.6B/bge-reranker-v2-m3.
- **Contextual retrieval** (prefixo de contexto por chunk) + **late chunking** (embedding do doc inteiro, depois recorte) para não perder contexto de voto longo.

Embeddings/rerank em escala

- **Qwen3-Embedding-0.6B** (Apache-2.0) na GPU em batch noturno; **4B** para reindex de alta precisão. Referência oficial (repo QwenLM/Qwen3-Embedding): *"The 8B size embedding model ranks No.1 in the MTEB multilingual leaderboard (as of June 5, 2025, score 70.58)"* — a 0,6B é o melhor custo/benefício para RAG multilíngue e Matryoshka permite truncar dimensão sem reindexar.
- Reranker Qwen3-Reranker sobre top-100 do estágio denso.

DataJud → modelos preditivos

- Extrair movimentações → dataset de desfecho (tempo até sentença, resultado, recursos) → **classificação + análise de sobrevivência (tempo até evento) + conformal prediction** (intervalos de confiança calibrados) para risco/êxito; risco contratual por NER de cláusulas + regras + LLM. Técnicas de referência: VICTOR (STF) e Athos (STJ) como precedente de classificação de peças/temas.
-

S3. Motor de inferência no 5060 Ti (sm_120)

Estado do suporte Blackwell consumer (sm_120) em vLLM (2026): kernels NVFP4 CUTLASS para SM120 já compilados (nvfp4_scaled_mm_sm120_kernels.cu , commit "Add NVFP4 MoE CUTLASS support for SM120");
cutlass_scaled_mm_supports_fp4(120)==True ; KV cache NVFP4 em SM120 via FlashInfer FA2 (PR #50288, com fix do V-scale swizzle para o path FA2 do SM120). Receita estável relatada: build vLLM recente + FlashInfer b12x + SM120 explícito + checkpoint ModelOpt NVFP4 + KV FP8 (não deixar o contrato NVFP4 do target "vazar" para o draft do speculative decoding). NVFP4 (bloco 16, escala FP8 E4M3) dá menor perplexidade que MXFP4 no mesmo modelo; é o default em Blackwell.

Tabela de cenários (single-stream, 5060 Ti 16 GB, 448 GB/s)

Cenário	VRAM aprox.	tok/s single	Observação
8B FP8	~9–10 GB	~75–95 (medido em 5060 Ti: 75 Ollama/llama.cpp; ~95 vLLM FP8)	folga p/ LoRAs + KV FP8
14B FP8/AWQ	~14–15 GB	~32–45 [est.]	KV FP8 obrigatório p/ contexto longo
27–32B EXL3 ~3 bpw	~13–15 GB	~18–30 [est.]	EXL3/TabbyAPI; melhor qualidade×bpw
Qwen3-30B-A3B / GPT-OSS-20B (MoE)	16 GB + offload	~30 (classe 35B-A3B relatada ~30 t/s c/ MTP em 16 GB)	MoE ativa ~3.3B/token
GPT-OSS-120B / Qwen3-235B-A22B	16 GB + 96–192 GB RAM	decode limitado por banda DDR5	ktransformers/llama.cpp offload

Contexto 32k–128k: KV FP8 (e NVFP4 KV onde estável) para caber; CAG: pré-carregar Constituição + códigos no KV como prefixo cacheado (automatic prefix caching do vLLM).

Frameworks e papéis

- vLLM (primário): FP8/NVFP4, automatic prefix caching, KV FP8, speculative decoding (EAGLE-3/MTP/n-gram/draft), sleep mode (libera GPU para treino

noturno), **LMCache** (offload de KV p/ RAM/NVMe), **Multi-LoRA** com base quantizada.

- **SGLang**: RadixAttention + FlashInfer em sm_120 (bom p/ árvores de prompt/estruturas jurídicas repetidas).
- **llama.cpp (build CUDA Blackwell)**: GGUF K-quants/IQ-quants, **multi-LoRA hot-swap por requisição** no `llama-server`, **offload de experts MoE p/ CPU** (`--n-cpu-moe / -ot`), KV Q8/Q4; **ik_llama.cpp** para kernels quant mais rápidos.
- **ExLlamaV3/TabbyAPI**: EXL3 2,5–4 bpw (trellis/QTIP), cache quant 2–8 bit, endpoint OpenAI.
- **TensorRT-LLM**: NVFP4 no limite de throughput (quando valer o custo de build).
- **ktransformers**: MoE grande híbrido com 96–192 GB RAM. **Sem AMX no Ryzen** → cai em AVX-512; decode é **memory-bandwidth bound**. Referência: ~16 tok/s Int4 em Xeon DDR5 usando ~50% da banda nominal (~221 GB/s efetivos). No 9800X3D com **DDR5-6000 dual channel** (~96 GB/s teórico), maximize com: single socket (NUMA irrelevante), threads = núcleos físicos, `-ot` para manter attention+KV na GPU e só experts na CPU. Alternativa emergente: **FreeToken** (UC Berkeley/MIT, ago/2026) faz split ótimo por camada em tempo real e rodou Qwen3.6-35B a ~39 tok/s num 4060 laptop de 8 GB [InfoQ](#) — vale acompanhar.
- **llama-swap (Go)**: roteador/hot-swap de modelos por endpoint.

Roteamento em cascata + saída estruturada

- **Cascata**: modelo pequeno (8B FP8) responde; se incerteza alta (entropia semântica / logprobs abaixo de limiar) → escala para 27–32B EXL3 ou MoE offloaded.
- **Cache semântico** de respostas verificadas (por embedding da pergunta) → hit devolve resposta + recibo sem tocar a GPU.
- **Gramática/saída estruturada**: **xgrammar** (vLLM) ou **llguidance/outlines** para forçar o schema de citação (URN, âncora, hash) — zero parsing frágil.

S4. Quantização agressiva

Estado da arte 2026 (2–4 bits em GPU consumer)

- **EXL3** (variante de **QTIP**: trellis + transformadas Hadamard aleatórias): near-FP16 em 6–8 bpw, **competitivo até 3 bpw**; bit-allocation por medição de KL-divergência por camada (`-l2 / -l3`); `-hb / -hq` para elevar bitrate de camadas sensíveis (attention/shared-expert). Melhor curva qualidade×bpw hoje para caber 27–32B em 16 GB. Comando de conversão:

```
python -m exllamav3.convert -i Qwen-32B-juridico -o Qwen-32B-exl3-3.0bpw
-w /tmp/exl3 -b 3.0 -hb 6 -hq
```

- **GGUF IQ2_M/IQ3_XXS/IQ4_XS** (i-quants): bom para offload MoE em llama.cpp.
- **AWQ/GPTQ** (~4–4.5 bpw efetivos): compatível com vLLM Multi-LoRA.
- **NVFP4/MXFP4** (Blackwell nativo): throughput; NVFP4 > MXFP4 em perplexidade.
- **Auxiliares**: HQQ/HQQ+, QTIP, QuIP#, AQLM/PV-tuning, SpinQuant/QuaRot (rotação p/ remover outliers).
- **KV**: KIVI/KVQuant/FP8 (e NVFP4 KV em sm_120).
- **Embeddings**: int8/binary/Matryoshka.
- **BitNet b1.58 + bitnet.cpp**: ternário {-1,0,+1}, ~0,4 GB (não-embedding) no 2B-4T; [ARGO](#) 100B a 5–7 tok/s em CPU single. No 9800X3D (x86, ganho bitnet.cpp 2,37–6,17× vs baseline; [GitHub](#) v2 de 15/01/2026 com kernels low-bit melhores) → **agentes 24/7 leves em CPU** (monitor de DJEN, classificador de ramo, watchdog) sem tocar a GPU.

Recuperação de qualidade pós-quantização

- **QAT / EfficientQAT / QA-LoRA / QLoRA sobre base quantizada; calibração com corpus jurídico pt-BR** (dataset de decisões do S2) — crítico: calibrar com o domínio, não com wikitext.
- **Meta prática**: 27–32B a ~3 bpw EXL3 inteiramente nos 16 GB; 14B FP8 com folga p/ LoRAs + KV; medir no **WikiJurídica-Bench** (precisão de citação, taxa de alucinação, além de OAB-Bench/Magis-Bench).

S5. Destilação e compressão (o núcleo do bot próprio)

Técnicas e por que cada uma

- **Logit KD** (KL forward/reverse, temperatura) — barato, denso.
- **Sequence-level KD** — treina em sequências completas do professor.
- **GKD** (TRL `GKDTrainer`) — mistura on/off-policy.
- **On-policy distillation** (Thinking Machines Lab, Kevin Lu, DOI 10.64434/tml.20251026, 27/10/2025): o aluno gera, o professor pontua **cada token** via KL reversa. É a técnica-espinha do pós-treino porque ensina o aluno a **recuperar dos próprios erros** (evita erro composto em cadeias longas de raciocínio jurídico), superando SFT. Números de referência: a Thinking Machines relata **70% em AIME'24 com 1.800 GPU-h vs 17.920 GPU-h** por RL; o Qwen3 Technical Report (arXiv:2505.09388, Tabela 21) mede, para o aluno Qwen3-8B, **AIME'24 74,4% com 1.800 GPU-h via on-policy distillation vs 67,6% com 17.920 GPU-h via RL** [Thinkingmachines](#) — ~10× menos compute com qualidade igual ou superior.
- **MiniLLM, DistiLLM-2, Arcee DistillKit, torchtune KD, Unsloth/Axolotl KD** — implementações práticas.

- **Destilação de raciocínio:** traces DeepSeek-R1/Qwen3-thinking → modelo pequeno (OpenThoughts/Open-R1). **Cuidado:** "small models struggle to learn from strong reasoners" — traces precisam ser filtrados/verificados por citação.
- **Multi-professor:** vários open-weight rodando por offload à noite como professores heterogêneos.
- **Self-distillation / strong-to-weak** (pipeline Qwen3).
- **Fusão heterogênea por destilação:** FuseLLM/FuseChat 3.0 (combinar famílias diferentes num aluno).
- **Poda: Minitron (NVIDIA)** prune+distill — Nemotron-Nano-9B-v2 destilou 12B→9B com **KD por KL forward** só na fase de recuperação, rodando 128k contexto em A10G 22 GB, [arXiv](#) 3–6× throughput de Qwen3-8B; [arXiv](#) Llama-3.1 8B→4B com só 400B tokens (40× menos custo de treino que treinar do zero).
ShortGPT/SliceGPT/LLM-Pruner/Wanda/SparseGPT p/ poda estruturada;
DistilBERT-style p/ auxiliares.

Receita executável no 5060 Ti (jurídico 4–8B)

1. **Professor:** 27–32B jurídico (merge do S6) rodando por offload noturno (vLLM sleep mode libera a GPU entre lotes) ou em llama.cpp com experts na CPU.
2. **Poda (Minitron):** de um 14B base, podar embedding/heads/FFN (l2-norm+mean, single-shot) [arXiv](#) para ~8B; depois profundidade (drop de subgrupo contínuo com menor queda em bench, ex.: Winogrande). [arXiv](#)
3. **Recuperação:** KD logit (KL forward) + **on-policy distillation** sobre prompts jurídicos reais (consultas de agentes do flywheel).
4. **Dados:** corpus S2 + Q&A sintético + **traces de raciocínio verificados por citação** — recompensa verificável = (URN resolve no banco canônico) E (modelo NLI confirma que o trecho sustenta a afirmação).
5. **VRAM/tempo [est.]:** QLoRA/KD de 8B em 16 GB cabe com batch pequeno + grad-checkpoint; tokens/h dependente de seq-len; janelas noturnas de 6–8 h com hot-swap ao amanhecer.

Test-time scaling do modelo pequeno

- **Budget forcing (s1), self-consistency, best-of-n com verificador/PRM jurídico, MCTS** para construção de argumentação, **tool-use RL (GRPO/RLVR)** em 16 GB via Unsloth (recompensa = citação verificada).

S6. Fusão, LoRA e "MoE de LoRAs"

MergeKit (Arcee, EMNLP 2024; >12 métodos)

Métodos atuais:

`slerp/nuslerp/linear/task_arithmetic/ties/dare_ties/della/model_stock/sce/karcher`

Karcher (média de Fréchet na esfera; PR #546, mar/2025) e **model_stock/sce/della** são os mais úteis para combinar especialistas por ramo. RAM: usar `--lazy-unpickle / --low-cpu-memory` para mergear 14B/32B com pouca RAM. Truque de calibração: **merge base+instruct com $\lambda \approx 0,4-0,5$** recupera calibração e diversidade (arXiv 2510.17426).

Exemplo YAML (merge de especialistas por ramo):

```
merge_method: della
base_model: juridico-14b-base
models:
  - model: lora-merged-civil
    parameters: {weight: 0.5, density: 0.7}
  - model: lora-merged-tributario
    parameters: {weight: 0.5, density: 0.7}
parameters: {epsilon: 0.1, lambda: 1.0}
dtype: bfloat16
```

"MoE de LoRAs" por ramo do direito

- **PEFT add_weighted_adapter** , **mergekit-extract-lora**, **LoraHub/Arrow/PHATGOOSE** (roteamento zero-shot de LoRAs).
- **X-LoRA**: gating denso de LoRAs = MoE sobre um único base (troca de especialista por token).
- **S-LoRA/Punica**: servir centenas de adaptadores num base — um LoRA por ramo (civil, penal, trabalhista, tributário, consumidor, família, previdenciário, administrativo, empresarial, constitucional) + roteador por **classificador de ramo** (BitNet/DistilBERT em CPU).
- **Evolutionary merging** (Sakana/Mergenetic) com avaliação local no WikiJurídica-Bench.

Treino (Unsloth/TRL/Axolotl/LLaMA-Factory) — VRAM/tokens-h em 16 GB

Tarefa	Modelo	Cabe em 16 GB?	Observação
QLoRA	8B	Sim, folgado	batch médio + grad-checkpoint
QLoRA	14B	Sim	batch pequeno, seq 2–4k
QLoRA	32B	Apertado	Unsloth offload; seq curto
GRPO/RLVR	8B	Sim (Unsloth)	recompensa por citação verificada
DPO/ORPO	8B–14B	Sim	preferência por qualidade de citação

Pipeline noturno: **treino** → **bench** → **canário** → **hot-swap**; **registry de adaptadores** com hashes SHA-256 e métricas (promoção só se supera o incumbente no WikiJurídica-Bench).

S7. Reuso de código aberto dentro do proprietário em Go

Componente	Ling.	Licença	Modo de reuso	Est
llama.cpp/ggml	C/C++	MIT	cgo (go-llama.cpp) ou sidecar llama-server	mé
Ollama (loader GGUF/runner)	Go	MIT	biblioteca/fork de módulo	mé
LocalAI	Go	MIT	sidecar/lib	bai
llama-swap	Go	MIT	binário roteador	bai
hugot	Go	Apache-2.0	biblioteca Go + ONNX Runtime	bai
onnxruntime_go	Go	MIT	biblioteca	bai
tokenizers (sugarme, daulet)	Go	Apache-2.0/MIT	biblioteca	bai
gomlx	Go	Apache-2.0	biblioteca	mé
MCP go-sdk (oficial)	Go	MIT	biblioteca	bai
mcp-go (mark3labs)	Go	MIT	biblioteca	bai
Eino / Genkit-Go / ADK-Go	Go	Apache-2.0	grafos de agentes	mé
River	Go	MPL-2.0	filas sobre Postgres	bai
NATS JetStream	Go	Apache-2.0	mensageria	bai
pgvector/pgvectorscale/AGE	C	PostgreSQL/Apache	extensão PG	bai

Componente	Ling.	Licença	Modo de reuso	Est
Bleve/Meilisearch/Qdrant	Go/Rust	Apache/MIT	lib/sidecar	bai
colly/rod	Go	Apache/MIT	biblioteca	bai
goldmark/templ/htmx	Go	MIT/BSD	biblioteca	bai
Aperture/LND (L402)	Go	MIT	sidecar/lib	mé
x402 (Go)	Go	Apache-2.0	biblioteca	bai
khatru/GoToSocial	Go	MIT/AGPL	sidecar protocolo social	mé
Docling/marker/Crawl4AI/SearXNG	Python	MIT/Apache	sidecar Python	mé
vLLM/SGLang	Python	Apache-2.0	sidecar (OpenAI API)	mé
Unsloth/TRL/MergeKit/DistillKit	Python	Apache/LGPL	ferramenta offline	mé

Ecossistema Go confirmado: o **MCP go-sdk oficial** (Anthropic, "in collaboration with Google") atingiu **v1.x estável**, com releases até **v1.7.0** trazendo suporte à revisão de protocolo **2026-07-28** (modelo stateless, `_meta` por requisição, `server/discover` substituindo o handshake `initialize`); [GitHub](#) o community **mcp-go (mark3labs)** chegou a **v1.0.0** com suporte à mesma revisão (importado por ~1.630 módulos). **hugot** roda embeddings/NER/rerank/classificação em ONNX **nativamente em Go** (backends ONNX Runtime, XLA e um backend Go puro em progresso) — usar para todos os auxiliares (NER pt-BR, reranker, classificador de ramo), eliminando o microserviço Python do hot path.

Arquitetura Go ↔ Python

- **gRPC** ou **Arrow Flight** (zero-copy p/ tensores/embeddings) ou **Unix domain sockets** para o hot path local.
 - Sidecars Python (vLLM, Docling) supervisionados por **systemd** com health-check HTTP; reinício automático; **GPU sharing** por MPS ou por janelas de tempo (scheduler do S10).
 - Go é o orquestrador/serving-front; Python só onde não há equivalente Go (treino, OCR VLM, serving de LLM grande).
-

S8. Camada M2M de pagamento e comércio entre agentes

Trilhos disponíveis (set/2026)

- **x402 (V2)**: revive HTTP 402; fluxo 402 + PAYMENT-REQUIRED (Base64) → cliente assina → PAYMENT-SIGNATURE → facilitador /verify + /settle → PAYMENT-RESPONSE com recibo. CAIP-2 para redes; USDC (EIP-3009) o caminho seamless. Doado à **x402 Foundation (Linux Foundation)** em abr/2026. Go oficial em `coinbase/x402/tree/main/go` (X402Client, ResourceServer, Facilitator, middleware net/http/Chi/Gin); [GitHub](#) alternativa community `mark3labs/x402-go` com suporte MCP. Escala real (Coinbase, via lançamento Agent.market 21/04/2026): "approximately 69,000 active AI agents on x402 have already processed over 165 million transactions totaling \$50 million in volume." [CryptoNews.com](#)
- **x402 Bazaar**: catálogo de serviços pagos descobríveis pelo CDP Facilitator; discovery público sem API key; endpoint MCP `api.cdp.coinbase.com/platform/v2/x402/discovery/mcp` com 10.000+ tools pagas. Publicar aqui é discovery grátis; a qualidade do listing usa volume de chamadas e pagadores únicos nos últimos 30 dias.
- **Cloudflare Pay-per-Crawl / Monetization Gateway**: 402 com header `crawler-price`; **preço mínimo \$0,001 por retrieval bem-sucedido** (Cloudflare: "Cloudflare sets a minimum price of \$0.001 per successful retrieval"); [Digital Boom](#) Cloudflare é Merchant of Record; liquida em USDC via x402; **Discovery API** (`crawlers-api.ai-audit.cfdata.org/charged_zones`, autenticada por Web Bot Auth, com o header de preço incluído na assinatura contra tampering). Já roda no seu Cloudflare — ligar direto. Contexto de escala: a Cloudflare relata que "customers across Cloudflare's network are sending more than one billion HTTP 402 Payment Required responses to AI crawlers per day" (escopo: 402 de AI-crawler, desde a GA de ago/2025). [Cloudflare](#) A partir de 15/09/2026 a Cloudflare bloqueia por padrão crawlers de treino/agente em páginas com anúncio de novos domínios — a WikiJurídica, como conteúdo estruturado próprio, define seu próprio preço.
- **L402/Apture (Lightning Labs, Go)**: proxy reverso HTTP 402 + macaroons + Lightning; **Apture é Go** (compila com Go 1.25+); Taproot Assets p/ stablecoins na Lightning. Bom para micropagamentos sub-centavo sem cadeia EVM.
- **Stripe MPP, Google AP2 (mandates) + extensão x402, Amazon Bedrock AgentCore Payments (buyer-side)** — interoperar como cliente quando o agente pagador usa esses.
- **RSL 1.0/TollBit** — licenciamento legível por máquina (declarar termos no `.well-known`).

Projeto do ledger (Go)

- Contas de agentes (chave Ed25519/DID), saldo pré-pago (USDC/sats), **metering** por tool/token/página, faturas 402 dentro de MCP e HTTP, **macaroons/scopes** (L402)

para autorização por capacidade, **carteira da entidade, reinvestimento automático** (regra: X% da receita → hardware, via S10).

- Precificação: **custo marginal por chamada = (kWh × tarifa RJ) + GPU-segundos**.
Ex.: consumo GPU ~180 W; 1 resposta de 2 s ≈ 0,0001 kWh; tarifa RJ residencial alta ~R\$0,90/kWh → **~R\$0,00009 de energia/resposta** → precificar em **US\$0,002–0,02** cobre energia + amortização com margem enorme.

Catálogo de produtos M2M

1. Páginas premium (inteiro teor estruturado + grafo).
2. **Análises de risco/êxito** (modelo preditivo do S2).
3. **Datasets/knowledge packs** (Parquet + embeddings alinhados ao índice).
4. **API de embeddings** alinhada ao índice (mesma dimensão/modelo → interoperável).
5. **Feeds de delta** diários.
6. **Ferramentas MCP** de previsão e de **verificação de citação** (/verify do S1).

Publicar no **MCP Registry**, **x402 Bazaar** e diretórios (ChatGPT Apps SDK, conectores do Claude, extensões do Gemini CLI, Copilot, Cursor/VS Code).

S9. Rede social para IAs (API-first)

- **Identidade**: chaves Ed25519/DIDs por agente; **Web Bot Auth (RFC 9421 HTTP Message Signatures)** para verificar agentes de provedores (mesmo mecanismo que a Discovery API da Cloudflare exige).
- **Registro/ações**: MCP tools (`register_profile` , `post` , `comment` , `debate_reply` , `react`) + REST espelhado.
- **Modelo de dados**: perfis (advogada/jurista/leiga IA), posts, threads, mídia, reações, **reputação por citações verificadas** (peso = citações que resolvem no banco canônico).
- **Tempo real**: SSE/WebSocket/NATS; **MCP resources/subscribe + A2A tasks** (`a2aproject/a2a-go` , spec v1.0 [GitHub](#) sob custódia da Linux Foundation) para debates longos assíncronos.
- **Moderação/juiz**: modelo local como **LLM-as-judge** (avalia procedência jurídica do argumento, não opinião).
- **Federação opcional**: Nostr (zaps Lightning ↔ L402) e ActivityPub via **GoToSocial**.
- **Pipeline de mídia local (NVMe + CDN)**: TTS pt-BR (XTTS-v2/F5-TTS/Kokoro/Orpheus); slides→vídeo via ffmpeg; avatar falante (LivePortrait/Hallo); **vídeo generativo em 16 GB: Wan 2.2 TI2V-5B** (Apache-2.0; FP8 cabe no GPU, 720p@24fps, até 832×480 e 81 frames; ~4 min/clip de 25 frames a 768×512 em

4090 [Will It Run AI](#) → [est. mais lento na 5060 Ti pela banda]; usar sequential offload); thumbnails Flux-schnell; transcrição faster-whisper.

Brainstorm de features inéditas (S9)

- **Debate estruturado com placar de citações:** cada afirmação num debate entre IAs deve vir com URN; o juiz local pontua por citações que resolvem + NLI. Leaderboard de "juristas IA" por procedência.
- **Moot court sintético:** duas IAs (autor/réu) + IA-juiz simulam um caso concreto extraído do DataJud; o desfecho previsto pelo modelo preditivo vira "gabarito".
- **Fact-check em tempo real:** todo post passa pelo `/verify`; posts sem citação válida recebem selo "não verificado".
- **Threads que viram answer units:** debate de alta reputação é destilado numa página canônica citável (alimenta S1 e o corpus de treino do S5).

S10. Sistema vivo 24/7

- **Loop de conteúdo (noturno):** ingestão diária (S2) → geração de "answer units" pelo modelo local em batch (vLLM sleep→wake) → **verificação de citação** (`/verify`) → publicação estática → **IndexNow** → notificação a assinantes (WebSub).
- **Flywheel:** consultas de agentes → detecção de lacunas (perguntas sem página) → geração de páginas → novos dados de treino → LoRA melhor → respostas melhores → mais citações.
- **Memória de longo prazo do agente:** episódica (últimas interações) + grafo (AGE) de entidades jurídicas.
- **Scheduler de GPU:** janelas — dia: serving (vLLM); madrugada: treino/KD (S5) + OCR batch (S2) + mídia (S9). Transições por vLLM sleep mode / llama-swap.
- **Auto-healing:** systemd + watchdog; reset de GPU (`nvidia-smi -r`) em travamento; reinício de sidecars por health-check.
- **Tesouraria autônoma:** regras — receita M2M → carteira → quando saldo > limiar, alocar para hardware (2ª GPU/RAM); alertas.
- **KPIs e promoção:** citações/dia (meta 100k), taxa de 304, custo por resposta, precisão de citação, taxa de alucinação. Promoção de modelo/adaptador **só** se supera incumbente em WikiJurídica-Bench + canário sem regressão.

S11. O que cada patamar de hardware desbloqueia

Upgrade	Desbloqueia	tok/s / capacidade [est.]
32→96 GB RAM	GPT-OSS-120B / Qwen3-235B-A22B por offload (ktransformers/llama.cpp)	decode limitado por banda DDR5; ~5–15 tok/s [est.]
96→192 GB RAM	Professores grandes simultâneos p/ multi-professor KD; KV/LMCache amplo	treino noturno mais rico
2ª RTX 5060 Ti (→32 GB, TP/PP)	32B FP8 em 2 GPUs; batch maior; ou 1 serving + 1 treino	~2× throughput agregado
RTX 3090 24 GB usada (936 GB/s)	melhor banda que 5060 Ti → decode single mais rápido; 32B mais folgado	banda ~2× → decode notavelmente melhor
RTX 5090 32 GB	32B NVFP4 confortável, contexto longo, mídia rápida	maior salto single-GPU
Nó memória unificada (Strix Halo 128 GB)	MoE grande com banda unificada p/ experts	alternativa a ktransformers
Cluster (llama.cpp RPC / vLLM Ray)	somar GPUs entre nó principal + secundário	escala horizontal própria

Recomendação: próximo passo = **RAM para 96 GB** (destrava MoE grande como professor de KD) e 2ª GPU **RTX 3090 24 GB usada** pela banda (936 GB/s) — melhor custo/decode que uma 2ª 5060 Ti. 5090 32 GB só se o orçamento permitir o salto.

S12. Roadmap de engenharia e handoff para o Cowork

0–30 dias (fundação de conteúdo + ingestão)

- Ativar dois planos: Cloudflare cache-everything + `Accept: text/markdown` + IndexNow + JSONL de deltas. Instrumentar Logpush→Postgres→Grafana com atribuição de referral.
- Ingestão DJEN (API pública) + STJ dados abertos + Planalto HTML; grafo AGE mínimo; índice híbrido pgvector + BM25 + Qwen3-Reranker-0.6B via hugot.
- vLLM em sm_120 com 8B FP8 + KV FP8; `/verify` com recibo Ed25519.
- **Bench a rodar:** tok/s 8B FP8 single/batch no 5060 Ti; páginas/hora Docling; precisão de citação inicial (WikiJurídica-Bench v0).

30–90 dias (bot próprio + M2M)

- Corpus S2 → calibração de quant jurídica; **EXL3 ~3 bpw de um 27–32B**; cascata pequeno→grande.
- **Destilação**: Minitron 14B→8B + on-policy distillation com recompensa por citação; registry de adaptadores.
- Ledger Go + x402 (Go) + Pay-per-Crawl ligado; publicar 3 produtos no x402 Bazaar + MCP Registry.
- **Bench**: WikiJurídica-Bench (citação/alucinação) do destilado vs base; custo/resposta medido.

90–180 dias (rede social + autonomia)

- API social IA (Ed25519 + Web Bot Auth + MCP tools); reputação por citação; moot court sintético.
- Pipeline de mídia (XTTS/Wan 2.2-5B) no scheduler noturno.
- "MoE de LoRAs" por ramo (S-LoRA/X-LoRA); flywheel fechado; tesouraria autônoma.
- Upgrade RAM 96 GB + 2ª GPU; professores grandes por offload para multi-professor KD.

Decisões default (para o Cowork assumir)

- Serving: **vLLM** (sm_120, FP8/NVFP4, KV FP8, prefix caching). Fallback MoE: **llama.cpp** com `-ot` experts→CPU.
- Quant de qualidade: **EXL3**; throughput: **NVFP4**.
- Pós-treino: **on-policy distillation** + Minitron; RLVR via Unsloth.
- Auxiliares: **hugot/ONNX em Go**.
- M2M: **x402 (Go) + Cloudflare Pay-per-Crawl + L402/Apture**; discovery via x402 Bazaar + MCP Registry.

Experimentos com comandos (para rodar no próprio 5060 Ti)

```
# vLLM 8B FP8 em sm_120 com KV FP8 e prefix caching
VLLM_ATTENTION_BACKEND=FLASHINFER vllm serve juridico-8b-fp8 \
  --quantization modelopt --kv-cache-dtype fp8 --enable-prefix-caching \
  --max-model-len 65536

# EXL3 3.0bpw de um 32B via TabbyAPI
python -m exllamav3.convert -i juridico-32b -o juridico-32b-exl3-3.0 -b
3.0 -hq

# llama.cpp MoE com experts na CPU (Qwen3-30B-A3B)
llama-server -m Qwen3-30B-A3B-Q4_K_XL.gguf -ngl 99 -ot
".ffn_.*_exps.=CPU" \
```

```
--cache-type-k q8_0 --cache-type-v q8_0 -c 32768

# BitNet agente 24/7 em CPU (9800X3D)
python setup_env.py -md models/BitNet-b1.58-2B-4T -q i2_s
```

Bibliografia (URLs reais, acesso 15/09/2026)

1. Cloudflare — Markdown for Agents (changelog 12/02/2026):
<https://developers.cloudflare.com/changelog/post/2026-02-12-markdown-for-agents/>
2. Cloudflare — Pay Per Crawl docs: <https://developers.cloudflare.com/ai-crawl-control/features/pay-per-crawl/what-is-pay-per-crawl/>
3. Cloudflare — Monetization Gateway (blog):
<https://blog.cloudflare.com/monetization-gateway/>
4. Cloudflare — Pay Per Crawl set a price (mínimo \$0,001):
<https://developers.cloudflare.com/ai-crawl-control/features/pay-per-crawl/use-pay-per-crawl-as-site-owner/set-a-pay-per-crawl-price/>
5. Coinbase — x402 Bazaar docs: <https://docs.cdp.coinbase.com/x402/bazaar>
6. x402 ROADMAP: <https://github.com/coinbase/x402/blob/main/ROADMAP.md>
7. Thinking Machines — On-Policy Distillation (27/10/2025):
<https://thinkingmachines.ai/blog/on-policy-distillation>
8. Qwen3 Technical Report (arXiv:2505.09388): <https://arxiv.org/abs/2505.09388>
9. Qwen3 Embedding (arXiv 2506.05176) e repo: <https://github.com/QwenLM/Qwen3-Embedding>
10. ExLlamaV3 convert docs: <https://github.com/turboderp-org/exllamav3/blob/master/doc/convert.md>
11. vLLM PR #50288 (NVFP4 KV SM120): <https://github.com/vllm-project/vllm/pull/50288>
12. NVIDIA Nemotron Nano 2 / Minitron (arXiv 2508.14444):
<https://arxiv.org/abs/2508.14444>
13. NVlabs/Minitron: <https://github.com/NVlabs/Minitron>
14. MergeKit merge_methods: https://github.com/arcee-ai/mergekit/blob/main/docs/merge_methods.md
15. hugot (ONNX em Go): <https://github.com/knights-analytics/hugot>
16. Microsoft BitNet / bitnet.cpp: <https://github.com/microsoft/BitNet>
17. Docling GPU: <https://docling-project.github.io/docling/usage/gpu/>
18. DJEN/Comunica API (CNJ): <https://www.cnj.jus.br/programas-e-acoes/processo-judicial-eletronico-pje/comunicacoes-processuais/orientacoes-aos-tribunais/>
19. STJ dados abertos: <https://dadosabertos.web.stj.jus.br/>
20. STF inteiro teor: <https://portal.stf.jus.br/jurisprudencia/pesquisarInteiroTeor.asp>

21. LexML SRU (Senado dados abertos): <https://www12.senado.leg.br/dados-abertos/legislativo/legislacao/acervo-do-portal-lexml>
 22. MCP Go SDK oficial: <https://github.com/modelcontextprotocol/go-sdk>
 23. A2A Go SDK: <https://github.com/a2aproject/a2a-go>
 24. Aperture (L402, Go): <https://github.com/lightninglabs/aperture>
 25. Wan 2.2 TI2V-5B: <https://huggingface.co/Wan-AI/Wan2.2-TI2V-5B>
 26. Wellows — estudo de citações de IA (jan–jun 2026): <https://wellows.com/blog/ai-citation-overlap-study/>
 27. ktransformers (SOSP'25):
<https://madsys.cs.tsinghua.edu.cn/publication/ktransformers-unleashing-the-full-potential-of-cpu/gpu-hybrid-inference-for-moe-models/SOSP25-chen.pdf>
-

Recomendações (staged, com gatilhos)

- **Agora (0–30d)**: commit nos dois planos e no `/verify` assinado. Gatilho de sucesso: taxa de 304 > 70% no tráfego de crawler e citações/dia subindo acima da baseline ~23k. Se 304 < 50%, revisar ETag/ `Vary` /purge por tag antes de qualquer coisa de GPU.
- **Curto (30–90d)**: destilar o primeiro jurídico 8B (Minitron+on-policy) e ligar x402/Pay-per-Crawl. Gatilho para promover o destilado a produção: superar o 14B FP8 base em precisão de citação no WikiJurídica-Bench **sem** regressão de alucinação no canário. Gatilho para EXL3 32B como professor default: caber ≤15 GB a ~3 bpw com perda de KL aceitável na calibração jurídica.
- **Médio (90–180d)**: rede social IA + flywheel fechado. Gatilho de upgrade de hardware: quando a fila de treino noturno não fecha em 8 h **ou** o serving diurno satura → comprar RAM 96 GB primeiro (destrava professor MoE grande), depois 3090 24 GB usada (banda). Só ir para 5090 32 GB se a receita M2M já pagar o salto pela tesouraria autônoma.
- **Contínuo**: cobrir os cinco motores separadamente (a sobreposição de 0,31% do estudo Wellows prova que otimizar para um só deixa ~99% da superfície de citação na mesa). Priorizar densidade de estatísticas + citações inline com URN (sinal GEO comprovado) sobre schema decorativo (sinal nulo no teste Ahrefs).

Caveats

- Números de tok/s do 5060 Ti vêm de benchmarks de blog/vendor (42→95 tok/s para 8B conforme backend/quant) e variam com FP16/FP8/Q4 e medido vs estimado — rodar o próprio bench antes de fixar SLA. Não há figura oficial NVIDIA.
- O suporte NVFP4/KV-NVFP4 em sm_120 no vLLM ainda dependia, em meados de 2026, de builds recentes de vLLM+FlashInfer e configuração explícita de SM120;

validar contra a versão exata do checkpoint e do runtime antes de produção.

- STF/STJ não expõem API REST oficial de busca textual de acórdãos; o caminho de inteiro teor é PDF (STF) e datasets CKAN (STJ) — a rota de SPA/JSON do STF não é API documentada e pode mudar. LexML/SRU entrega metadados + link, não a íntegra (esta vem do publicador, tipicamente Planalto).
- A transferência do x402 para a x402 Foundation/Linux Foundation e a data (abr/2026) têm parte da confirmação em fonte secundária (blog); o repo/releases oficiais são a autoridade para versão. Métricas de escala do x402 (~69k agentes, \$50M) são autorrelato Coinbase.
- A janela DDR5 do ktransformers no 9800X3D é limitada por banda (dual channel ~96 GB/s teórico) e pela ausência de AMX no Zen 5 — o decode de MoE grande será modesto; tratar como capacidade de professor/batch noturno, não de serving interativo de baixa latência.